Unidad 1 / Escenario 2

Lectura fundamental

Arquitectura de referencia y estilos arquitectónicos de *software*

Contenido

1 Arquitectura de referencia

2 Estilos arquitectónicos de *software*

Palabras clave: model-Driven-Architecture, agilidad, modelos, perspectivas.

1. Arquitectura de referencia

La arquitectura de referencia alude a una estructura prediseñada, estandarizada o consensuada que optimiza el trabajo, aumenta la confiabilidad y propicia una implementación efectiva y eficaz a todo nivel (empresarial, *software*, sistemas) en cualquiera de los dominios de la arquitectura (negocio, aplicaciones, infraestructura tecnológica; dinámico, estático, creacional). Las arquitecturas de referencia son soluciones que incorporan mejores y más actualizadas tecnologías técnicas y prácticas para el diseño de *software*.

1.1. ¿Por qué debo considerar una arquitectura de referencia, cuáles son los beneficios?

Con esta herramienta el arquitecto simplifica el diseño arquitectónico y ofrece resultados más ágiles, lo cual no significa rapidez, aunque esté relacionada, pero si eficiencia y eficacia en la elaboración de la arquitectura. El mercado requiere que el *software* se implemente de manera eficiente y debido a esto, con frecuencia, eligen arquitecturas de referencia.

Puede no ser el camino rápido, pero si el más corto: la arquitectura de referencia no necesariamente ahorra tiempo, pero sí permite que el arquitecto centre su trabajo en características que son importantes, en la adaptación de las soluciones existentes y no en la reinvención de soluciones ya probadas. Otra ventaja es la posibilidad de diseñar de manera estandarizada, lo que hace posible manejar un mismo lenguaje, facilitando la comunicación y la reutilización.

Por tal razón, como arquitecto debe entender que el proceso de diseño de *software* requiere una inversión considerable de tiempo y experiencia. Si bien es posible diseñar una solución de *software* sin arquitecturas de referencia, no considerarla puede significar una desventaja enorme y retrocesos.

1.2. ¿Qué pasos debo seguir para crear una arquitectura de referencia?

Las arquitecturas de referencia se diseñan, construyen y se validan utilizando pruebas exhaustivas antes de ser documentadas por arquitectos expertos. Debido a que las arquitecturas de referencia se implementan en muchas situaciones, los expertos incorporan ajustes a partir de experiencias reales, buscando la satisfacción de los drivers de arquitectura, en versiones posteriores a la arquitectura de referencia (Dell EMC, 2018).

Esto sugiere un proceso de mejora continua; los pasos a seguir son: planear, hacer, verificar y actuar (PHVA) de manera constante e iterativa. En consecuencia, la empresa se beneficia de la sabiduría acumulada a través de estos pasos o el PHVA de la arquitectura de referencia (ver figura 1).

Figura 1. PHVA Elaboración de una arquitectura de referencia

Fuente: elaboración propia

Las arquitecturas de referencia están elaboradas de manera simple, sencilla y modular. Son desarrolladas a través de una vista y descritas a través del punto de vista que define un vocabulario común con el que se puede discutir los detalles de implementación para un producto de *software*. Esta estructura permite a los clientes crear una solución que incluye componentes adaptados a sus drivers de arquitectura (Dell EMC, 2018).

1.3. Diferencia entre los marcos de referencia y la arquitectura de referencia

En palabras cortas, un marco de referencia guía el cómo y el qué, mientras que la arquitectura de referencia solo trata el qué. Esta describe el *software*, mientras que un marco guía describe el proceso de construcción de la arquitectura y sugiere distintas vistas y puntos de vista u otro artefacto; además, agrupa-ordena estos artefactos o componentes en niveles, vistas o incluso arquitecturas (ver tabla 1). Un artefacto es un elemento tangible resultante del proceso de arquitectura puede ser código, pero usualmente es un documento, puntos de vista, matices, inventarios.

Tabla 1. Resumen de vistas-artefactos propuestos por marco de referencia

Marco de referencia	Tipo de agrupación	Vistas o artefactos propuestos
Zachman	Niveles	Scope, empresa, **sistema lógico**, tecnología, representación, funcionamiento.
Togaf	Arquitecturas	Arquitecturas
4+1	Vistas (conjunto de Artefactos)	**Lógica, proceso**, física, desarrollo, casos de uso.
Grady Booch, James Rumbaugh e Ivar Jacobson	Vistas (conjunto de artefactos)	**Diseño, proceso,** implementación, despliegue, casos de uso.
POSA	Vistas (conjunto de artefactos)	**Lógica, proceso,** física, desarrollo.
Microsoft	Vistas (conjunto de artefactos)	Lógica, **conceptual**, física.

Fuente: Elaboración propia (2018). Modificado de Reynoso (2004)

1.4. Modelos

No es posible desarrollar grandes sistemas o *software* sin diseño y tampoco diseño sin arquitectura; es necesario establecer modelos que permitan una adecuada descripción de manera que apoyen su estudio y evaluación del diseño. Según Olivier Gerbé (HEC Montréal) y Jean Bézivin, profesor emérito de l'Université de Nantes (universidad pública francesa), un modelo "es una simplificación de un sistema construido con un objetivo previsto en mente. El modelo debería ser capaz de responder preguntas en lugar del sistema real " (Bézivin & Gerbé, 2001).

Los modelos pueden describir cualquier cosa, incluso a sí mismos; en consecuencia, se pueden tener modelos en cualquiera de los niveles de abstracción que se quiera, permitiendo establecer, a partir de ellos, arquitecturas de referencia. Entre los modelos más conocidos están:

- Model-Driven Software (MDS).
- Model-Driven Engineering (MDE)
- Model-Driven Architecture (MDA)
- Model-Driven Software Modernization (MDSM)

1.5. ¿Qué relación tienen los modelos y metamodelos?

Un modelo es una imagen que representa un sistema (Brown, Conallen & Tropeano, 2005), es decir, una simplificación (completa y consistente) del sistema real, que sirve para comprenderlo mejor. Cuando la descripción hace referencia al modelo y no al sistema, entonces se establece la relación jerárquica entre los dos modelos, el que describe al sistema y el que se describe a sí mismo; este último se conoce como metamodelo. Así, un metamodelo es simplemente el modelo de un modelo. Los modelos o metamodelos pueden tener una representación gráfica o textual de un sistema (o de sí mismos en el caso del metamodelo).

1.6. ¿Cuál es su origen y destino?

Los modelos y metamodelos quizás no sean la última ni la mejor herramienta para describir un sistema o a sí mismos. Los paradigmas y artefactos cambian aproximadamente cada 15 años (ver figura 2), pero es útil y lo seguirá siendo. Los modelos cada vez toman más relevancia en el proceso de desarrollo de *software*. Según Bézivin (2014), la construcción de *software* ha evolucionado de plantear que "todo es un objeto" a finales del siglo pasado, a proponer que "todo es un modelo" a comienzos de este siglo.

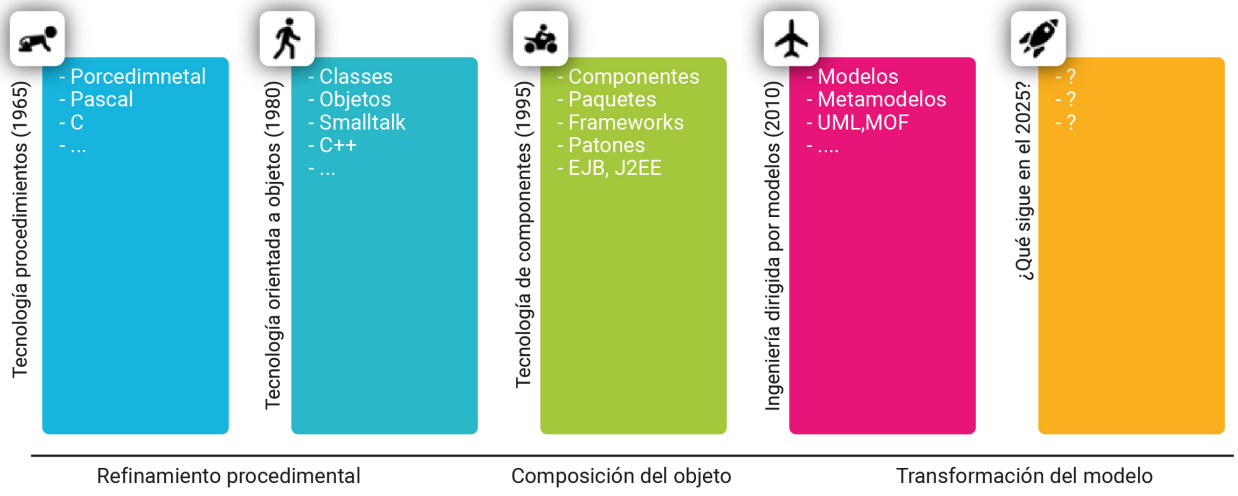


Figura 2. Modelos de SW, cambio de paradigma y/o artefacto.

Fuente: Elaboración propia (2018). Modificado de Bézivin (2014)

En una empresa de desarrollo de *software*, en el contexto académico o en el desarrollo personal existe un proceso natural de adopción de la ingeniería dirigida por modelos. La siguiente figura presenta las fases y la relación entre el modelo y el código.

Figura 3. Fases del proceso de adopción del paradigma dirigido por modelos

Fuente: Elaboración propia (2018). Modificado de Beydeda, Book y Gruhn (2005)

1.7. Perspectivas

Dependiendo de los intereses, la academia, la industria y los vendedores tienen distintos puntos de vista y tan solo al primero de ellos le interesa describirlos. La perspectiva académica difiere de la de los otros dos sectores en que estos manejan una mirada del desarrollo dirigida por modelos y cada uno de ellos usa lenguajes con sus propios enfoques, que son más o menos formales. En contraste, en el mundo de la academia se reconoce la formalidad y, si se hace algo por fuera de ella, se protocoliza mediante la estructuración del alcance o la definición del punto de vista; empero, en el caso de las grandes industrias y los vendedores esto es demasiado subjetivo.

Así pues:

En el ámbito académico: la descripción de la arquitectura se basa en técnicas, estilos y lenguajes específicos (ADL: Architecture Description Language), que se abordarán en el desarrollo del módulo.

En el ámbito de los grandes vendedores: su descripción está sesgada por la tecnología específica que desarrollan (JEE, .NET, IBM).

En el caso de los industriales: generan puntos de vista híbridos o adaptados según las distintas tecnologías y frameworks utilizados; existe una integración de vistas correspondiente con el número de tecnologías implementadas. Difícilmente tienen una oficina de arquitectura, lo que conlleva a una mezcla heterogénea de arquitecturas, vistas y puntos de vista.

¿Sabía qué...?

No solo la arquitectura aplica para el mundo de *software*, sino para TI en las empresas. Colombia cuenta con un marco de referencia que es el principal instrumento para implementar la arquitectura TI del país y habilitar la estrategia de Gobierno en línea. Con él se busca habilitar las estrategias de TIC para los servicios, la gestión, el gobierno abierto, la seguridad y la privacidad. (Mintic, s.f.)

1.8. Técnicas de modelado según perspectiva

Dependiendo de las perspectivas anteriormente mencionadas, existe una técnica de modelado que se usa con mayor frecuencia en cada caso: en la academia, es ADL o Architecture Description Language; para los grandes vendedores son los diagramas libres; y en el ámbito industrial, los diagramas de paquetes y, en general, el desarrollo dirigido por modelos (perfiles UML).

1.9. Herramientas CASE (Computer Aided Software Engineering)

Las herramientas de Ingeniería de Software Asistida por Computadora son diversas aplicaciones informáticas o programas informáticos destinados a aumentar la productividad en el proceso de ingeniería de *software*, especialmente en el diseño y la arquitectura.

Estas herramientas ofrecen varias ventajas: realizan modelado, diagramación, corrección sintáctica, verifican coherencia entre diagramas y hacen integración con otras aplicaciones; algunas herramientas en línea ofrecen trabajo multiusuario y otras más avanzadas permiten la reutilización y generación de código. En la tabla 2 se presentan algunas de ellas.

Tabla 1. Resumen de vistas-artefactos propuestos por marco de referencia

Nombre	url	Uso
Enterprise Architect	http://sparxsystems.com	Formal – generar código
TAU UML Suite	https://www.ibm.com/*software*/products/es/ratitau	Formal– generar código
cacoo	https://cacoo.com/	Libre (Formal-informal)
Argo UML	http://argouml.tigris.org/	Formal– generar código
lucidchart	https://www.lucidchart.com	Libre (Formal-informal)
Altova UModel	https://www.altova.com/umodel	Formal– generar código
MagicDraw UML	https://www.nomagic.com/	Formal– generar código

Fuente: Elaboración propia (2018). Modificado de Reynoso (2004)

2. Estilos arquitectónicos de *software*

El término estilo arquitectónico a menudo, erróneamente, es usado como sinónimo de patrón, y esto se debatirá durante mucho tiempo porque tienen más características en común que diferencias, entre otras, tener un nombre que los identifica y solucionar un asunto recurrente en el diseño de sistemas. A continuación, se define lo que es y lo que no es el estilo arquitectónico a través de definiciones de algunos autores.

Lo que no es

Los estilos solo se manifiestan en arquitectura teórica descriptiva de alto nivel de abstracción; no se ocupan del diseño detallado, algoritmos y estructuras de datos. El patrón es una solución a bajo nivel, por eso, para este módulo, un estilo no es un patrón.

Lo que es

El estilo de una arquitectura es una taxonomía (clasificación) de las configuraciones posibles, tiene un carácter descriptivo (estructura) y heurístico (descubridor) (Reynoso y Kicillof, 2004). Se le considera descubridor en el sentido que establece una estructura que permite definir la mejor arquitectura a través de su evaluación continua.

> *Un estilo arquitectónico define una familia de sistemas en términos de patrón de organización estructural. Más específicamente, determina el vocabulario de los componentes y conectores que se pueden usar en instancias de ese estilo, junto con un conjunto de restricciones sobre cómo pueden ser conjunto.*
> *(Garlan & Shaw, 1994)*

Por otro lado, Reynoso y Kicillof (2004) afirman que "un estilo describe entonces una clase de arquitectura, o piezas identificables de las arquitecturas empíricamente dadas. Esas piezas se encuentran repetidamente en la práctica, trasuntando la existencia de decisiones estructurales coherentes".

El referirse al estilo arquitectural, subraya explícitamente la existencia de una taxonomía de las alternativas estilísticas. Puntualmente, las alternativas más comunes de estilos son: cliente-servidor, modelo-vista-controlador, tubería-filtros y arquitectura en capas (Shaw y Garlan, 1996). Algunos de ellos se presentan a continuación.

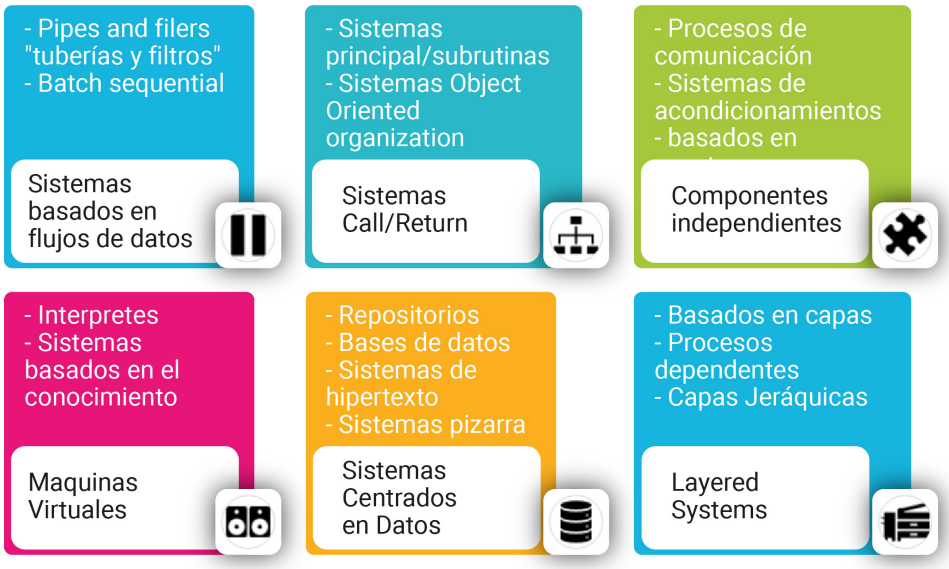


Figura 4. Clasificación de los estilos arquitectónicos

Fuente: Elaboración propia (2018). Modificado de Shaw & Garlan (1996)

Cómo mejorar...

La práctica hace al maestro; en la medida en que los desarrollos de *software* incluyan ejercicios de arquitectura acordes y notaciones formales, ejemplo UML, se podrá adquirir mayor experiencia y se reducirá el tiempo de diseño. El uso constante de estilos le permitirá concentrarse en los drivers importantes, más que en la forma de implementar la solución.

2.1. Tubería y filtros (Pipes and filters)

Este estilo arquitectural se relaciona con el flujo de datos y las redes de proceso, y es uno de los menos ambiguos. Una tubería (pipeline) une componentes computacionales (filtros) a través de conectores (pipes), con el fin de realizar transformaciones sucesivas (process). Los filtros tienen comúnmente la tarea de eliminar campos o registros, pero no es la única que realiza (ver figura 5). Las líneas discontinuas son entradas y salidas del sistema mientras que las flechas son tuberías.

Figura 5. Ejemplo de un estilo arquitectural de tubería y filtros

Fuente: Elaboración propia (2018). Modificado de Shaw & Garlan (1996)

2.2. Modelo orientado a objetos (Data Abstraction and Object-Oriented Organization)

Este estilo presenta los elementos esenciales (administradores) y las relaciones entre los elementos (llamadas) que se ejecutan mediante interfases-instancias de ellos (invocaciones).

Algunas de las características más relevantes se presentan a continuación y, según el caso, se podrán ver como ventaja o desventaja.

- Un objeto es responsable de preservar la integridad de la representación (usualmente manteniendo algunos datos invariables)
- La representación está oculta para otros objetos
- Los componentes principales son objetos
- Es posible remplazar la implementación, ya que está oculta o no es totalmente dependiente.

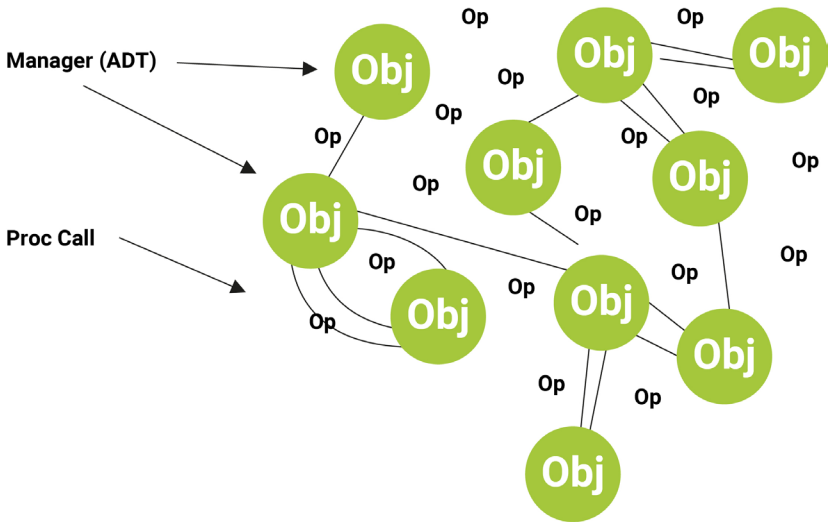


Figura 6. Abstract Data Types and Objects.

Fuente: Elaboración propia (2018). Modificado de Shaw & Garlan (1996)

En la figura anterior, obj es un administrador y op es una invocación. Existen variaciones del estilo; por ejemplo, algunos sistemas pueden permitir a los "objetos" realizar tareas concurrentes, otras múltiples interfases y otras una mezcla de todo.

2.3. Cliente–servidor

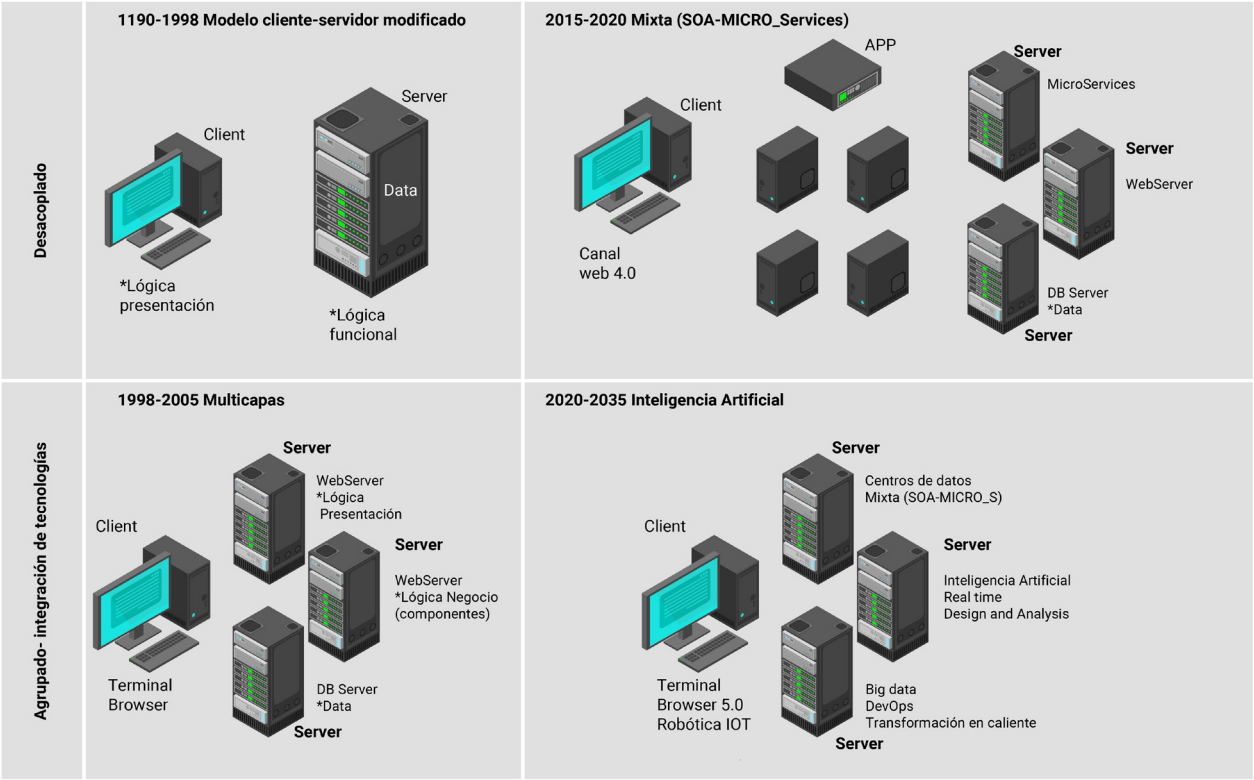
Este estilo realiza una agrupación de funciones de cómputo (datos-lógica y presentación) de manera abstracta, razón por la cual también se le nomina Arquitectura de dos niveles. Esta describe un sistema distribuido que involucra un nodo central (el servidor) y varios nodos (los clientes) que se comunican con el primero a través de la red.

Históricamente, las aplicaciones clientes-servidor se componían de una aplicación de escritorio que se comunicaba con un DBMS o con un servidor de archivos dedicado.

Los DBMS contenían gran parte de la lógica de negocio en forma de procedimientos almacenados. A través del tiempo, la agrupación de funciones de cómputo oscila entre el cliente y el servidor de manera consecuente con las mejoras tecnológicas a nivel de hardware. Hoy en día tenemos computadoras (celulares) más poderosas en nuestros bolsillos de lo que eran los DBMS.

Tabla 3. Comportamiento elástico y cíclico, arquitectura cliente-servidor

Tipo	Ciclo 1	Ciclo 2
Acoplamiento fuerte	**1970-1980 Modelo Host** Client Server *Lógica presentación *Lógica funcional *Data	**2005- 2009 Modelo SOA/BPM** Client Canal web 2.0 BPM ESB Portal DashBoard Server WebServer *Lógica Presentación Server WebServer *Lógica Negocio (componentes) DB Server *Data Server
Acoplamiento debil	**1980-1990 Modelo cliente-servidor dos niveles** Client Server Data Lógica presentación *Lógica funcional	**2009-2015 Micro servicios** Client Canal web 3.0 APP APP Server *Lógica presentación *Lógica funcional *Data Server *Lógica presentación *Lógica funcional *Data *Lógica presentación *Lógica funcional *Data Server



Fuente: elaboración propia

La tabla 3 realiza una descripción de un comportamiento elástico y cíclico de la configuración arquitectónica en el que se evidencia que, dada la evolución constante de la tecnología, la carga de trabajo o funciones se reparte de manera oscilante entre el cliente y el servidor. Este comportamiento tiene una periodicidad o se hace más evidente en una razón aproximada de 20 años con intervalos de 5 años entre cada una de las 4 fases (acoplamiento fuerte, acoplamiento débil, desacoplado, agrupación o integración de tecnologías).

2.4. Tres capas

En la tabla 3 también se puede evidenciar una agrupación comúnmente utilizada, que separa la lógica, la presentación y las bases de datos o persistencia. A esta separación se le conoce como estilo arquitectural de 3 capas. Un sistema organizado por capas conserva una relación cliente-servidor en cada una de sus capas; de esta manera, las capas de mayor jerarquía o superiores pueden interactuar con una inferior a través de servicios.

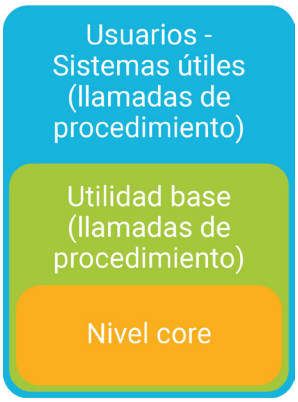


Figura 7. Estilo de arquitectura por capas

Fuente: Elaboración propia (2018). Modificado de Shaw & Garlan (1996)

En síntesis...

El estilo es útil y característico; la arquitectura es cuestión de estrategia y hasta para eso hay que tener estilo. Optar por una forma arquitectónica no solamente define una situación pragmática, sino que tiene valor distintivo. No es necesario reinventar la rueda; es pertinente conocer las distintas opciones y diferenciarlas para poder proponer mejoras y seleccionar la más adecuada según el contexto.

Referencias

Brown A.W., Conallen J. & Tropeano D. (2005). *Introduction: Models, Modeling, and Model-Driven Architecture (MDA).* En S. Beydeda, M. Book y V. Gruhn (Eds.). Model-Driven Software Development (pp.1-16). Berlin, Heidelberg: Springer.

Bézivin, J. & Gerbé, O. (Noviembre de 2001). *Towards a Precise Definition of the OMG/MDA Framework. Proceedings 16th International Conference on Automated Software Engineering.* IEEE Xplore, San Diego.

Dell EMC. (2018). *Glosario de Dell EMC*. Recuperado de https://colombia.emc.com/corporate/glossary/reference-architecture.htm

Garlan, D. & Shaw, M. (1994). *An Introduction to Software Architecture.* Pittsburgh, EUA: School of Computer Science, Carnegie Mellon University.

Mintic. (s.f.). *Arquitectura TI Colombia. Marco de Referencia.* Recuperado de http://www.mintic.gov.co/arquitecturati/630/w3-propertyvalue-8114.html

Reynoso, C. y Kicillof, N. (2004). *Estilos y Patrones en la Estrategia de Arquitectura de Microsoft. Versión 1.0*. Buenos Aires, Argentina: Universidad de Buenos Aires. Recuperado de http://148.202.167.116:8080/xmlui/bitstream/handle/123456789/940/Estilos%20y%20patrones%20en%20la%20estrategia%20de%20arquitectura%20de%20Microsoft.PDF?sequence=1

Referencias de imágenes y tablas

Beydeda, S., Book, M., & Gruhn, V. (2005). *Model-Driven Software Development.* Berlin: Springer.

Bézivin, J. (2014). *Software modelling and the future of engennering. 7th International Conference on Model Transformation.* ICMT, York (UK).

Shaw, M. & Garlan, D. (1996). *Software Architecture: Persperctives on an emerging dicipline.* New Jersey: Prentice Hall.

Reynoso, C. B. (2004). *Introducción a la Arquitectura de Software.* Buenos Aires, Argentina: Universidad de Buenos Aires. Recuperado de http://carlosreynoso.com.ar/archivos/carlos-reynoso-introduccion-a-la-arquitectura-de-*software*.pdf

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Arquitectura del software

Unidad 1: Introducción y conceptos generales sobre arquitecturas del software

Escenario 2: Arquitectura de referencia y estilos arquitectónicos del software

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Alejandro Torres

Asistente: Leidy Alejandra Morales Eslava

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.